

Hoeffding Adaptive Tree Regressor

Teoria, Implementazione in MOA/CapyMOA e Validazione Sperimentale

Stream Data Analytics 2025–2026

Andrea Zhang

Politecnico di Milano

2026

- 1 Motivazione
- 2 Background Teorico
- 3 Scelte di Design e Risultati
- 4 Dettagli Implementativi
- 5 Conclusioni

Perché implementare hatr in CapyMOA?

Il problema

River offre una implementazione di riferimento di HATR, ma non esiste come learner nativo in **MOA/CapyMOA**, il framework di riferimento per il benchmarking in streaming.

I ricercatori che usano **CapyMOA** non possono confrontarsi con HATR senza cambiare framework.

L'obiettivo

Portare HATR in MOA come learner nativo, numericamente equivalente a **River**, con:

- wrapper Python con API compatibile con **River**
- validazione sperimentale completa su 8 dataset
- codice pronto per il contributo upstream a MOA

Vantaggi strutturali di CapyMOA

- **Speedup JVM**: $3-8\times$ più veloce di River (dettagli nella sezione 3)
- **Schema ARFF**: tipi degli attributi dichiarati automaticamente — nessun bisogno di specificare `nominal_attributes`
- **Ecosistema MOA**: evaluator, dataset, confronto con altri learner MOA/CapyMOA
- **Footprint in memoria**: modello $3-10\times$ più compatto rispetto a River

CapyMOA vs. River: panoramica dei vantaggi

Perché la JVM è più veloce?

Java viene JIT-compilato a codice nativo; Python esegue bytecode interpretato. Per operazioni numeriche intensive (alberi, statistiche online) il vantaggio è strutturale.

Schema ARFF: nessuna configurazione manuale

Con **River**: l'utente deve dichiarare `nominal_attributes` per ogni feature categorica, altrimenti vengono trattate come numeriche.

Con **CapyMOA**: lo schema ARFF dichiara il tipo di ogni attributo. `inst.attribute(i).isNominal()` funziona in automatico.

Ecosistema MOA

Accesso diretto a: evaluator prequential, dataset ARFF standard, confronto con FIMTDD, ARF, kNN, ecc.

Hoeffding Tree: base per lo streaming

Hoeffding Tree (VFDT) [1] è un albero decisionale online che costruisce split usando la **Hoeffding bound**:

$$\varepsilon(n, \delta) = \sqrt{\frac{\ln(1/\delta)}{2n}}$$

Regola di split: dati i due migliori attributi a_1, a_2 (per variance reduction):

$$\text{split se } \frac{\Delta \bar{G}(a_2)}{\Delta \bar{G}(a_1)} < 1 - \varepsilon \quad \text{oppure} \quad \varepsilon < \tau$$

Grace period g : si tenta lo split ogni g istanze per ammortizzare il costo.

Split criterion — Variance Reduction

$$\Delta \bar{G}(a) = \text{Var}(S) - \sum_v \frac{|S_v|}{|S|} \text{Var}(S_v)$$

- S = istanze nel nodo
- S_v = sottoinsieme che va al ramo v
- Var = varianza campionaria ($ddof=1$, Welford)

Limitazione

Un Hoeffding Tree standard non rileva i **concept drift**: una volta creato, un nodo non viene revisionato.

ADWIN: finestra adattiva per il drift

ADWIN (ADaptive WINdowing) [2] mantiene una **finestra di dimensione variabile** sullo stream di errori.

Idea: si cerca il taglio $W = W_0 \cup W_1$ che massimizza la differenza di medie; se questa supera la soglia, la parte vecchia viene eliminata.

Criterio di taglio (Babcock et al., 2003 — variance-aware):

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| > \varepsilon_{cut}$$

$$\varepsilon_{cut} = \sqrt{\frac{2 m^{-1} \hat{\sigma}_W^2 \delta'}{1}} + \frac{2}{3} m^{-1} \delta'$$

dove $\delta' = \ln(2 \ln(|W|)/\delta)$, $m^{-1} = \frac{1}{|W_0|} + \frac{1}{|W_1|}$.

Struttura a bucket



Ogni riga i contiene bucket di dimensione 2^i ; quando una riga è piena, il *compress* fonde i due bucket più vecchi nella riga successiva.

Meccanismo adattivo di hatr

Ogni **nodo interno** (branch) di HATR porta:

- 1 un rilevatore ADWIN che monitora $|y - \hat{y}|$
- 2 un **albero alternativo** in background

Ciclo di vita a ogni istanza $(\mathbf{x}, y)$:

- Predici dal nodo corrente; calcola errore
- Aggiorna ADWIN
- **Se drift rilevato** e l'errore *sta aumentando*: crea albero alternativo (foglia) alla stessa profondità
- **Se albero alternativo maturo** ($n_{\text{alt}} > w_{\text{min}}$): esegui **z-test**
- Se z-test significativo: *swap* o *pruning*

Z-test per lo swap

Confronta l'errore medio del path corrente (μ_c, σ_c^2) con quello dell'alternativo (μ_a, σ_a^2):

$$z = \frac{\mu_a - \mu_c}{\sqrt{\sigma_a^2/n_a + \sigma_c^2/n_c}}$$

$$p = 2 \Phi(-|z|)$$

- $p \leq \alpha$ e $\mu_a < \mu_c$: **swap** (alternativo è migliore)
- $p \leq \alpha$ e $\mu_a \geq \mu_c$: **pruning** (corrente è migliore)

Predizione (durante la fase di switch): media delle foglie del path corrente e dell'alternativo.

Modalità di foglia

HATR supporta tre strategie di predizione alle foglie:

MEAN

Predice con la **media target** accumulata nella foglia (aggiornamento incrementale).

$$\hat{y} = \bar{y}_{\text{leaf}}$$

Semplice, robusto, deterministico.

MODEL

Predice con un **modello lineare online** (*Perceptron*, gradiente su perdita quadratica):

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

$$\nabla = 2(\hat{y} - y) [\mathbf{x}; 1]$$

I figli ereditano una **copia** del modello del padre.

ADAPTIVE (default)

Sceglie dinamicamente tra MEAN e MODEL tramite **faded MSE** (EWMA):

$$e_{\text{mean}} \leftarrow \gamma e_{\text{mean}} + (y - \bar{y})^2$$

$$e_{\text{model}} \leftarrow \gamma e_{\text{model}} + (y - \hat{y}_m)^2$$

Usa MEAN sse $e_{\text{mean}} < e_{\text{model}}$.

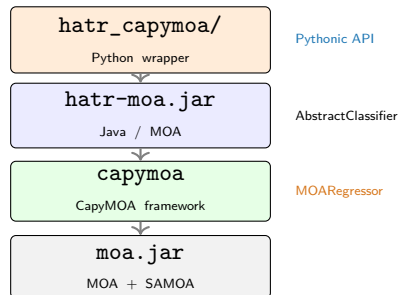
Bootstrap sampling opzionale ($\text{Poisson}(\lambda = 1)$) in tutte le modalità per aumentare la diversità delle foglie.

Architettura del progetto

L'obiettivo era portare HATR di **River** in **CapyMOA** come learner nativo MOA, mantenendo la stessa gerarchia di classi e gli stessi default.

Il progetto si divide in tre parti:

- `hatr-moa/` — classi Java nel package `moa.classifiers.trees.hatr`
- `hatr_capymoa/` — wrapper Python che espone `HoeffdingAdaptiveTreeRegressor(MOAREgressor)`
- `experiments/` — notebook di validazione `River↔CapyMOA`



Default preservati da River

<code>splitter</code>	<code>TEBSTSplitter</code> → <code>HATEBSTObserver</code>
<code>drift_detector</code>	<code>ADWIN($\delta=0.002$)</code> → <code>ADWINDetector</code>
<code>leaf_prediction</code>	<code>"adaptive"</code> → <code>AdaLeafAdaptive</code>
<code>leaf_model</code>	<code>LinearRegression()</code> → <code>HAPerceptron</code>
<code>grace_period</code>	200 <code>delta</code> <code>1e-7</code> <code>tau</code> <code>0.05</code>
<code>bootstrap</code>	<code>True</code> <code>switch_sig.</code> <code>0.05</code> <code>min_split</code> <code>5</code>

Scelte di design: configurazione implementata

Cosa è stato implementato

Solo la configurazione default di River:

- Splitter: HATEBSTObserver (Truncated E-BST)
split binari per attributi numerici
- Drift detector: ADWINDetector
- Leaf model: HAPerceptron (SGD lineare)
- `binary_split=False` hardcoded
attributi nominali → multiway branch

14/16 parametri River sono configurabili nel wrapper Python.

Motivazione

La configurazione default copre la grande maggioranza dei casi d'uso pratici. L'equivalenza numerica con **River** è verificabile e garantita su tutti gli 8 dataset testati.

Cosa non è stato implementato

- `binary_split=True`
split nominale binario forzato (non-default in River)
- `QOSplitter`
richiederebbe `AdaNumMultiwayBranch` per attributi numerici
- **Componenti pluggabili**
splitter, drift detector e leaf model sono fissi al default River

Stato	Esempio
Configurabile	<code>grace_period</code> , <code>delta</code> ✓
Hardcoded default	<code>splitter</code> , <code>ADWIN</code> ~
Non implementato	<code>binary_split=True</code> ✗

Setup sperimentale

Dataset ($n = 17\,000$, prequential evaluation):

Dataset	Feat.	Drift
Synthetic	5	abrupt
Bike Sharing	12	naturale
Fried	10	nessuno
Hyper (lento)	10	graduale
HyperFast	10	rapido
FriedDrift	10	abrupt
RBFReg	10	graduale
ElecDemand	7	naturale

Cap a 17 000 ist. Bike/ElecDemand: reali.
Hyper/HyperFast: generatore.

Configurazioni:

- *Deterministic*: mean, no bootstrap
- *Default*: adaptive, bootstrap=True

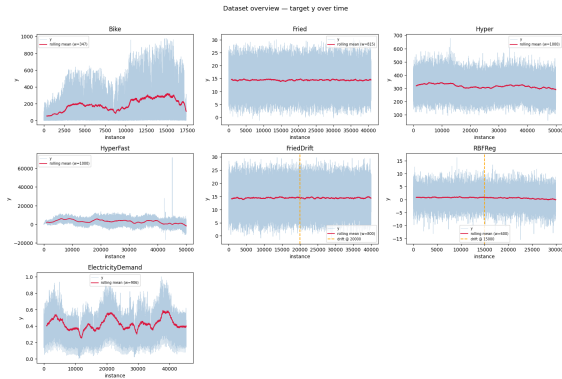
Iperparametri comuni:

- `grace_period=50`, $\delta = 10^{-7}$, $\tau = 0.05$
- `adwin_delta=0.002`,
`switch_significance=0.05`
- `drift_window_threshold=300`
- `model_selector_decay=0.95`
- `learning_ratio=0.01`, `seed=0`
- StandardScaler online identico ai due lati

Notebook:

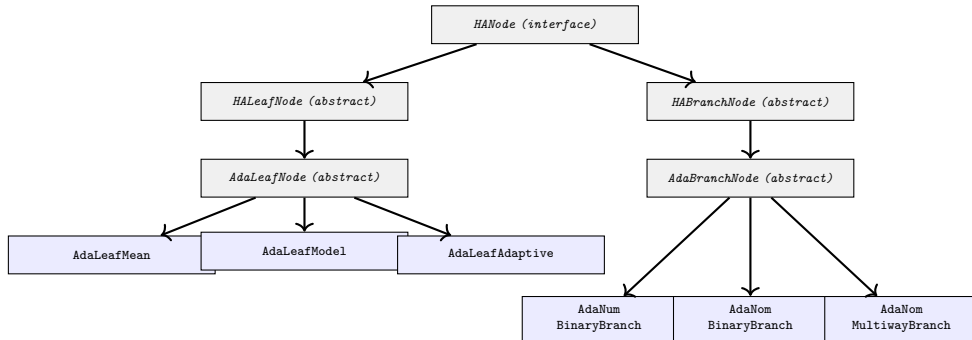
- `hatr-comparison.ipynb`: equivalenza
River↔CapyMOA
- `hatr-crossframework-quality.ipynb`:
MAE/RMSE/ R^2
- `hatr-crossframework-performance.ipynb`:
throughput
- `tutorial_04ext`: scenari di drift

Dataset: panoramica caratteristiche



Caratteristiche statistiche degli 8 dataset usati negli esperimenti.

Struttura Java: gerarchia dei nodi



Tipi di branch (split)

Tipi di branch (split)

- **AdaNumBinaryBranch**

split numerico: $x_i \leq t$; soglia da E-BST / TE-BST

- **AdaNomBinaryBranch**

split nominale binario: $x_i = v$ vs. tutto il resto

- **AdaNomMultiwayBranch**

un ramo per valore categorico; valore non visto $\rightarrow$ nuovo figlio dedicato (come River `add_child`); `maxBranches()` = -1

Perché non AdaNumMultiwayBranch?

In River esiste `AdaNumMultiwayBranchReg`, ma viene usata *solo* con `QOSplitter(allow_multiway_splits=True)`, configurazione non-default. Il default splitter (`TEBSTSplitter`) produce sempre split binari per attributi numerici $\Rightarrow$ `AdaNumBinaryBranch` è sufficiente per l'equivalenza con River HATR standard.

binary_split non implementato come opzione

River espone `binary_split=False` (default): per attributi nominali il multiway viene preferito; il binario lo sostituisce solo se ha merit strettamente superiore. Il Java replica questo comportamento in modo fisso in `HANominalObserver`: `binary_split=True` non è supportato.

Componenti di supporto

Observer per attributi numerici

- HAEBSTObserver — E-BST standard
- HATEBSTObserver — **Truncated** E-BST con arrotondamento *half-even* (`BigDecimal.HALF_EVEN`) per replicare `round(x, digits)` di Python

Utilities auto-contenute

- VarStats — varianza incrementale Welford (port di `river.stats.Var`)
- ADWINDetector — port fedele da River Cython
- HAPerceptron — modello lineare SGD
- NormalDist — CDF via Apache Commons Math 3 (`Erf.erfc`)

Dipendenze Maven

moa (scope: provided) +
commons-math3:3.6.1 (scope: provided, già nel moa.jar)

Dettagli implementativi non ovvi

Truncated EBST e arrotondamento

Il TEBST arrotonda i valori di split a un numero fisso di cifre decimali, riducendo i candidati unici nell'albero. Python `round()` usa banker's rounding (half-even); `Math.round()` in Java usa half-up. La discrepanza rompeva l'equivalenza: risolto usando `BigDecimal(x).setScale(d, HALF_EVEN)`.

Predizione con albero alternativo attivo

`traverseWithAlternate` non viene chiamato solo al momento dello swap: durante *tutta la vita* dell'albero alternativo, la predizione è la media tra il path principale e quello alternativo.

Attributi nominali: River vs. MOA

River — schema-less

Le feature arrivano come dizionari Python senza tipo dichiarato. `nominal_attributes` (lista di nomi) è l'unico modo per indicare al tree quali feature trattare come categoriche; senza, verrebbero passate al `TEBSTSplitter` numerico.

MOA — schema ARFF

Ogni dataset porta uno schema che dichiara il tipo di ogni attributo. `inst.attribute(i).isNominal()` funziona in automatico: nessuna configurazione richiesta all'utente.

merit_preprune: quando ha effetto?

Meccanismo

Con `merit_preprune=True` il candidato *null split* ($\text{merit} = -\infty$) viene aggiunto alla lista. Se vince il test di Hoeffding la foglia viene **disattivata**.

Il *null split* viene selezionato solo se `len(candidates) < 2` (unico candidato). Con `len ≥ 2`, il controllo `best.merit > 0` fallisce prima ancora di raggiungere la selezione del candidato — il *null* non viene mai scelto.

Effetto pratico nullo con feature continue

Con feature numeriche continue (TEBSTSplitter), ogni observer produce sempre almeno un candidato $\Rightarrow \text{len} \geq 2$ garantito dopo `grace_period`. Il *null split* ($\text{merit} = -\infty$) non può mai essere il migliore: `merit_preprune` è **ininfluente** nel caso tipico, come confermato sperimentalmente (River e CapyMOA identici con `True` e `False`).

merit_preprune: quando ha effetto?

Quando ha effetto concreto

River applica `min_samples_split` nella split criterion: se un branch ha pochi campioni $\Rightarrow$ `merit=0` (candidato creato, non scartato).

- `preprune=False`, feature nominale ad alta cardinalità: candidato con `merit=0` $\Rightarrow$ `len=1` $\Rightarrow$ **split forzato** anche senza segnale
- `preprune=True`: aggiunge null $\Rightarrow$ `len=2` $\Rightarrow$ richiede `merit>0` $\Rightarrow$ **nessun split spurio**

Verificato sperimentalmente: 60 foglie (False) vs. 1 foglia (True).

Classe principale: HoeffdingAdaptiveTreeRegressor

```
public class HoeffdingAdaptiveTreeRegressor
    extends AbstractClassifier {

    // MOA Option system
    public IntOption gracePeriodOption
        = new IntOption("gracePeriod", 'g', ..., 200);
    public FloatOption deltaOption
        = new FloatOption("delta", 'd', ..., 1e-7);
    public MultiChoiceOption leafPredictionOption
        = new MultiChoiceOption("leafPrediction", 'l',
            new String[]{"MEAN", "MODEL", "ADAPTIVE"},
            2);
    // ...

    public HANode root;    // radice dell'albero

    @Override
    public void trainOnInstanceImpl(Instance inst) {
        if (root == null) {
            root = newLeaf(null, 0); nActiveLeaves = 1;
        }
        ((AdaLeafNode/AdaBranchNode) root)
            .adaLearnOne(inst, this, null, -1);
    }
}
```

```
@Override
public double[] getVotesForInstance(Instance
    inst) {
    // Raccoglie foglie dal path principale
    // E da tutti gli alberi alternativi
    incontrati
    List<HALeafNode> leaves =
        ((AdaBranchNode) root)
            .traverseWithAlternate(inst);
    double pred = 0;
    for (HALeafNode l : leaves)
        pred += l.getPrediction(inst);
    return new double[]{pred / leaves.size()};
}

public void attemptToSplit(
    HALeafNode leaf, HABranchNode parent,
    int parentBranch, ADWINDetector branchDet
    ) {
    // Hoeffding bound -> crea branch adattivo
    // lastSplitAttemptAt = child.
    getTotalWeight()
}
}
```

Memory management

Ogni 10^6 istanze (configurabile), la classe principale esegue un ciclo di stima e controllo della memoria:

- ❶ **Stima** (`estimateModelByteSizes`): misura la RAM reale via `SizeOf` e aggiorna le stime per foglia attiva e inattiva
- ❷ **Controllo** (`enforceTrackerLimit`): se l'albero supera `maxByteSize`, ordina le foglie per *promise* (profondità) e **disattiva** le meno promettenti finché si rientra nel limite
- ❸ **Potatura observer** (`pruneLeafObservers`): dopo ogni split, rimuove dagli EBST i tagli con merit troppo basso rispetto al migliore

Foglia attiva vs. inattiva

Una foglia **inattiva** smette di aggiornare i propri attribute observer (risparmio di RAM), ma continua ad aggiornare la statistica target e a predire da essa. Non può più splittare.

Se il budget lo permette, viene **riattivata**.

Opzione `stopMemManagement`

Se attiva, al superamento del limite si blocca direttamente `growthAllowed` invece di disattivare foglie: nessun nuovo split viene tentato.

`SizeOf` richiede l'agente JVM

`-javaagent:sizeofag.jar`; se assente, il meccanismo è silenziosamente disabilitato.

Cuore adattivo: AdaBranchNode.adaLearnOne()

```
public void adaLearnOne(Instance inst, HATR tree, HABranchNode parent, int parentBranch) {
    double y = inst.classValue();
    double yPred = traverseToLeaf(inst).getPrediction(inst);    // main tree only
    stats.update(y, inst.weight());

    double err = Math.abs(y - yPred);
    double oldMean = errorTracker.getMean();
    driftDetector.update(err);                                // ADWIN update
    errorTracker.update(err, 1.0);
    boolean driftOccurred = driftDetector.isDrift();

    if (driftOccurred && errorTracker.getMean() < oldMean)    // errore in calo, skip
    { errorTracker = new VarStats(); driftOccurred = false; }

    if (driftOccurred && alternateTree == null) {            // CREA albero alternativo
        alternateTree = tree.newLeaf(null, depth); tree.nAlternateTrees++;
    } else if (alternateTree != null) {
        VarStats altErr = getErrorTracker(alternateTree);
        if (altErr.getN() > tree.driftWindowThreshold ...) {
            double z = (altErr.getMean() - errorTracker.getMean()) /
                Math.sqrt(altV/altN + curV/curN);
            double p = 2.0 * NormalDist.cdf(-Math.abs(z));
            if (p <= tree.switchSignificance) {
                if (altErr.getMean() < errorTracker.getMean())
                    { /* SWAP */ } else { /* PRUNING */ }
            }
        }
    }
    // Propaga ad albero alternativo E al figlio corrente
    dispatch(alternateTree, inst, tree, ...);
    dispatch(next(inst), inst, tree, ...);
}
```

Python wrapper hatr_capymoa

```
class HoeffdingAdaptiveTreeRegressor(MOAREgressor):
    """Port nativo MOA di River's HATR.
    Numericamente equivalente senza bootstrap."""

    def __init__(self, schema, grace_period=200,
                  split_confidence=1e-7, leaf_prediction="adaptive",
                  bootstrap_sampling=True, adwin_delta=0.002,
                  learning_ratio=0.01, random_seed=None, ...):

        leaf = leaf_prediction.lower()
        cli = [
            f"-g {grace_period}",
            f"-d {split_confidence}",
            f"-l {_LEAF_PREDICTION[leaf]}",
        ]
        if bootstrap_sampling: cli.append("-b")
        if merit_preprune: cli.append("-p")

        self.moa_learner = _MOA_HATR()
        super().__init__(schema=schema,
                         CLI=" ".join(cli), random_seed=random_seed,
                         moa_learner=self.moa_learner)
```

Uso (dalla documentazione):

```
from capymoa.datasets import Fried
from hatr_capymoa import (
    HoeffdingAdaptiveTreeRegressor)
from capymoa.evaluation import (
    prequential_evaluation)

stream = Fried()
learner = HoeffdingAdaptiveTreeRegressor(
    schema=stream.get_schema(),
    leaf_prediction="adaptive",
    bootstrap_sampling=True)

results = prequential_evaluation(
    stream, learner, max_instances=10000)
print(results["cumulative"].rmse())
```

Random seed

randomSeedOption è protected in AbstractClassifier (non visibile da JPyype)
→ usare `h.setRandomSeed(0)` prima di `prepareForUse()`.

Mapping parametri River → CapyMOA

I default del wrapper Python corrispondono a River.

River	flag MOA	Default
grace_period	-g	200
delta	-d	1e-7
tau	-t	0.05
leaf_prediction	-l	adaptive
model_selector_decay	-e	0.95
bootstrap_sampling	-b	True
min_samples_split	-m	5
drift_window_threshold	-w	300
switch_significance	-s	0.05
max_depth	-D	None (illim.)
merit_preprune	-p	True
remove_poor_attrs	-R	False
max_size	-M	500 MB
stop_mem_management	-S	False

Parametri non esposti nel wrapper

- `leaf_model` — hardcoded `HAPerceptron`; lr configurabile via `learning_ratio` (-L)
- `splitter` — hardcoded `HATEBSTObserver`; digits via `tebst_digits` (-z)
- `drift_detector` — hardcoded `ADWINDetector`; delta via `adwin_delta` (-A)
- `binary_split` — hardcoded `False`
- `nominal_attributes` — non necessario (schema ARFF di MOA)
- `seed` — via `random_seed`

Compatibilità API: River → CapyMOA HATR

Parametro River	CapyMOA
grace_period	grace_period ✓
delta	split_confidence ✓
tau	tie_threshold ✓
leaf_prediction	leaf_prediction ✓
model_selector_decay	model_selector_decay ✓
bootstrap_sampling	bootstrap_sampling ✓
min_samples_split	min_samples_split ✓
drift_window_threshold	drift_window_threshold ✓
switch_significance	switch_significance ✓
max_depth	max_depth ✓
merit_preprune	merit_preprune ✓
remove_poor_attrs	remove_poor_attrs ✓
stop_mem_management	stop_mem_management ✓
seed	random_seed ✓

Parametro River	Stato
<i>Hardcoded al default River:</i>	
splitter	fisso TEBST ~
drift_detector	fisso ADWIN ~
leaf_model	fisso Perceptron ~
<i>Gestito automaticamente:</i>	
nominal_attrs	schema ARFF ✓
<i>Non implementato:</i>	
binary_split	solo False ✗

✓ 14/16 parametri pienamente supportati.

~ 3 componenti hardcoded al default River.

✗ 1 opzione non-default non implementata.

La configurazione standard (binary_split=False, TEBSTSplitter, ADWIN, Perceptron) è **completamente replicata** e numericamente equivalente.

Completamento parametri non-default

- 1 **binary_split=True**
Modificare HANominalObserver per imporre split binari anche su attributi nominali; aggiungere flag -B al learner MOA e parametro nel wrapper Python.
- 2 **Splitter pluggabile + AdaNumMultiwayBranch**
Implementare AdaNumMultiwayBranchReg e un HAQObserver per supportare QOSplitter(allow_multiway_splits=True); esporre splitter come opzione MOA.
- 3 **Leaf model pluggabile**
Aggiungere supporto per modelli leaf alternativi al Perceptron (es. regressione ridge).

Estensioni di architettura

- 4 **Drift detector pluggabile**
Definire un'interfaccia HADriftDetector e supportare detector alternativi ad ADWIN (es. EDDM, Page-Hinkley).

Riferimenti bibliografici I

- [1] P. Domingos and G. Hulten.
Mining high-speed data streams.
KDD, 2000.
- [2] A. Bifet and R. Gavaldà.
Learning from time-changing data with adaptive windowing.
SDM, 2007.
- [3] A. Bifet and R. Gavaldà.
Adaptive learning from evolving data streams.
IDA, 2009.
- [4] B. Babcock, M. Datar, R. Motwani, and L. O'Callaghan.
Maintaining variance and k-medians over data stream windows.
PODS, 2003.

Riferimenti bibliografici II

- [5] H. M. Gomes *et al.*
Adaptive random forests for evolving data stream classification.
Machine Learning, 2017.
- [6] E. Ikonovska, J. Gama, and S. Dzeroski.
Learning model trees from evolving data streams.
Data Mining and Knowledge Discovery, 2011.
- [7] A. Bifet *et al.*
MOA: Massive Online Analysis.
JMLR, 2010.
- [8] C. Grzenda *et al.*
CapyMOA: A Python library for data stream mining.
arXiv, 2024.

Grazie per l'attenzione

Codice, notebook e JAR disponibili in:
`2025-2026_Regression-Models-in-CapyMOA/andrea_zhang/`

Implementazione:	<code>implementation/hatr-moa/</code>
Wrapper Python:	<code>implementation/hatr_capymoa/</code>
Esperimenti:	<code>experiments/</code>
Tutorial:	<code>tutorials/</code>